

Multi-Model Cascade Router

Eigenständiges druckbares Deep-Dive-Dossier

Multi-Model Cascade Router

Ich wollte nicht alles blind durch ein Modell schicken. Der Router behandelt Modelle als Ressourcen mit Stärken, Kosten, Quoten, Ausfällen, Qualität und Datenschutzkontext.

- Zeigt Modell-Governance in der Praxis: Modelle werden nach Kosten, Qualität, Privacy, Quoten, Ausfallrisiko und Eskalationslogik gewählt, nicht nach Bauchgefühl.
- Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.

Was ich mir dabei gedacht habe

Ich wollte Modellwahl nicht als Geschmack behandeln. Routing sollte Teil der Architektur sein: günstig, wenn möglich; stark, wenn nötig; lokal, wenn Privacy zählt; fallback-fähig, wenn ein Provider ausfällt.

Was ich gebaut habe

- Ich habe Modellwahl als Governance-System behandelt, nicht als Geschmacksfrage.
- Ich habe Kosten, Latenz, Quoten, Qualität und Fallbacks im selben System gedacht.
- Ich habe lokale und API-basierte Modelle in einer Routing-Logik verbunden.

Mechanik im Detail

- Der Router behandelt Modelle als Infrastrukturressourcen mit Stärken, Kosten, Quoten, Cooldowns, Privacy-Profilen und Ausfallmodi.
- Die Task-Strategie entscheidet, ob Geschwindigkeit, Qualität, kostenlose Modelle, lokale Ausführung, Code-Fähigkeit oder bezahlte Eskalation wichtiger ist.
- Der Sicherheitswert liegt in graceful degradation: Wenn ein Modell oder eine Ebene ausfällt, existiert eine Policy für den nächsten Weg.

Architektur

- Task-Strategien wählen schnell, hochwertig, gratis, lokal, Code, kreativ oder bulk.
- Lokale Modelle und kostenlose APIs werden dort bevorzugt, wo es sinnvoll ist.
- Quoten, Cooldowns und Ausfälle blockieren schlechte Routen.
- Qualitätsgates entscheiden, ob eskaliert wird.
- Mehrere freie Modelle können gegeneinander laufen, langsamere Calls werden abgebrochen.

Evidenzcluster

- AI-Router mit Provider-Registry für lokale Modelle, freie APIs, bezahlte/CLI-Modelle und LiteLLM-gestützte Calls.
- Cascade-Logik mit lokalen/freien/bezahlten Ebenen, Quoten, Cooldowns, Quality Gates, Free-Model-Racing, Cancellation und Budgetgrenzen.
- LiteLLM-Konfiguration mit getrennten lokalen, freien API- und bezahlten Ebenen.

- Architekturnotizen zu Routing-Prinzipien, Qualitätsbewertung, Budgethandling und fallbacks bei Quoten/Ausfällen.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt Modell-Governance in der Praxis: Modelle werden nach Kosten, Qualität, Privacy, Quoten, Ausfallrisiko und Eskalationslogik gewählt, nicht nach Bauchgefühl.	AI-Router mit Provider-Registry für lokale Modelle, freie APIs, bezahlte/CLI-Modelle und LiteLLM-gestützte Calls.; Cascade-Logik mit lokalen/freien/bezahlten Ebenen, Quoten, Cooldowns, Quality Gates, Free-Model-Racing, Cancellation und Budgetgrenzen.	Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	Task-Strategien wählen schnell, hochwertig, gratis, lokal, Code, kreativ oder bulk.; Lokale Modelle und kostenlose APIs werden dort bevorzugt, wo es sinnvoll ist.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt Modell-Governance in der Praxis: Modelle werden nach Kosten, Qualität, Privacy, Quoten, Ausfallrisiko und Eskalationslogik gewählt, nicht nach Bauchgefühl.

- Provider-neutrales Denken: Kein einzelnes Modell wird als magisch behandelt.
- Governance-Denken: Kosten, Qualität, Privacy und Quoten gehören zur Modellwahl.
- Messpfad: Outcome Logging schafft die Grundlage für spätere empirische Routing-Entscheidungen.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Multi-Model Cascade Router.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.

- Ein guter Agent muss elegant schlechter werden koennen, wenn Modelle ausfallen.
- Qualitätsgates sollten Eskalation auslösen, nicht Modell-Prestige.
- Outcome-Logging ist der Anfang empirischer Modell-Governance.

Nächste Härtung / nächster Projektschritt

- Frische Benchmarks ueber repraesentative Aufgabenklassen laufen lassen.
- Aggregierte Kosten/Latenz/Qualität ohne Geheimnisse veröffentlichen.
- Policy-Labels für privat, öffentlich, billig, schnell und hochriskant einfuehren.
- Benchmark mit synthetischen Aufgaben: Code Repair, Research Summary, Planung, Long-Context Extraction und Low-Cost-Bulk-Routing.
- Aggregierte Latenz/Kosten/Qualität ohne private Provider-Details veröffentlichen.
- Policy-Labels ergänzen, damit hochriskante oder private Aufgaben auf sicherere Routen gezwungen werden können.